

Documentation

Introduction

What ⚡⚡

Developer-to-developer documentation can be in one of two forms:

1. **Documentation for *developer-as-user*:** Software components are written by developers and reused by other developers, which means there is a need to document how such components are to be used. Such documentation can take several forms:
 - API documentation: APIs expose functionality in small-sized, independent and easy-to-use chunks, each of which can be documented systematically.
 - Tutorial-style instructional documentation: In addition to explaining functions/methods independently, some higher-level explanations of how to use an API can be useful.

- 📖 Example of API Documentation: [String API](#).
- 📖 Example of tutorial-style documentation: [Java Internationalization Tutorial](#).

2. **Documentation for *developer-as-maintainer*:** There is a need to document how a system or a component is designed, implemented and tested so that other developers can maintain and evolve the code. Writing documentation of this type is harder because of the need to explain complex internal details. However, given that readers of this type of documentation usually have access to the source code itself, only *some* information needs to be included in the documentation, as code (and code comments) can also serve as a complementary source of information.

- 📖 An example: [se-edu/addressbook-level4 Developer Guide](#).

Another view proposed by Daniele Procida in [this article](#) is as follows:

There is a secret that needs to be understood in order to write good software documentation: there isn't one thing called documentation, there are four. They are: tutorials, how-to guides, explanation and technical reference. They represent four different purposes or functions, and require four different approaches to their creation. Understanding the implications of this will help improve most software documentation - often immensely. ...

TUTORIALS A tutorial: • is learning-oriented • allows the newcomer to get started • is a lesson Analogy: teaching a small child how to cook	HOW-TO GUIDES A how-to guide: • is goal-oriented • shows how to solve a specific problem • is a series of steps Analogy: a recipe in a cookery book
EXPLANATION An explanation: • is understanding-oriented • explains • provides background and context Analogy: an article on culinary social history	REFERENCE A reference guide: • is information-oriented • describes the machinery • is accurate and complete Analogy: a reference encyclopedia article

Software documentation (applies to both user-facing and developer-facing) is best kept in a text format for ease of version tracking. **A writer-friendly source format is also desirable** as non-programmers (e.g., technical writers) may need to author/edit such documents. As a result, formats such as Markdown, AsciiDoc, and PlantUML are often used for software documentation.

Guidelines

Guideline: Go top-down, not bottom-up

What ⚡⚡

When writing project documents, a top-down breadth-first explanation is easier to understand than a bottom-up one.

Why ⚡⚡⚡

The main advantage of the top-down approach is that the document is structured like an upside down tree (root at the top) and **the reader can travel down a path she is interested in until she reaches the component she is interested to learn in-depth**, without having to read the entire document or understand the whole system.

How ⚡⚡

📖 To explain a system called `SystemFoo` with two sub-systems, `FrontEnd` and `BackEnd`, start by describing the system at the highest level of abstraction, and progressively drill down to lower level details. An outline for such a description is given below.

[First, explain what the system is, in a black-box fashion (no internal details, only the external view).]

`SystemFoo` is a

[Next, explain the high-level architecture of `SystemFoo`, referring to its major components only.]

`SystemFoo` consists of two major components: `FrontEnd` and `BackEnd`.

The job of `FrontEnd` is to ... while the job of `BackEnd` is to ...

And this is how `FrontEnd` and `BackEnd` work together ...

[Now you can drill down to `FrontEnd`'s details.]

`FrontEnd` consists of three major components: `A`, `B`, `C`

`A`'s job is to ...

`B`'s job is to...

`C`'s job is to...

And this is how the three components work together ...

[At this point, further drill down to the internal workings of each component. A reader who is not interested in knowing the nitty-gritty details can skip ahead to the section on `BackEnd`.]

In-depth description of `A`

In-depth description of `B`

...

[At this point drill down to the details of the `BackEnd`.]

...

Guideline: Aim for comprehensibility

What ⚡⚡

Technical documents exist to help others understand technical details. Therefore, **it is not enough for the documentation to be accurate and comprehensive; it should also be comprehensible.**

How ⚡⚡⚡

Here are some tips on writing effective documentation.

- **Use plenty of diagrams:** It is not enough to explain something in words; complement it with visual illustrations (e.g. a UML diagram).
- **Use plenty of examples:** When explaining algorithms, show a running example to illustrate each step of the algorithm, in parallel to worded explanations.
- **Use simple and direct explanations:** Convolved explanations and fancy words will annoy readers. Avoid long sentences.
- **Get rid of statements that do not add value:** For example, 'We made sure our system works perfectly' (who didn't?), 'Component X has its own responsibilities' (of course it has!).
- **It is not a good idea to have separate sections for each type of artifact,** such as 'use cases', 'sequence diagrams', 'activity diagrams', etc. Such a structure, coupled with the indiscriminate inclusion of diagrams without justifying their need, indicates a failure to understand the purpose of documentation. Include diagrams when they are needed to explain something. If you want to provide additional diagrams for completeness' sake, include them in the appendix as a reference.

Guideline: Document minimally, but sufficiently

What ⚡⚡

Aim for 'just enough' developer documentation.

- Writing and maintaining developer documents is an overhead. You should try to minimize that overhead.
- If the readers are developers who will eventually read the code, the documentation should complement the code and should provide only just enough guidance to get started.

How ⚡⚡⚡

Anything that is already clear in the code need not be described in words. Instead, **focus on providing higher level information that is not readily visible in the code or comments.**

Refrain from duplicating chunks of text. When describing several similar algorithms/designs/APIs, etc., do not simply duplicate large chunks of text. Instead, **describe the similarities in one place and emphasize only the differences in other places.** It is very annoying to see pages and pages of similar text without any indication as to how they differ from each other.

Tools

JavaDoc

What ⚡⚡

JavaDoc is a tool for generating API documentation in HTML format from comments in the source code. In addition, modern IDEs use JavaDoc comments to generate explanatory tooltips.

📖 An example method header comment in JavaDoc format:

```
1  /**
2   * Returns an Image object that can then be painted on the screen.
3   * The url argument must specify an absolute {@link URL}. The name
4   * argument is a specifier that is relative to the url argument.
5   * <p>
6   * This method always returns immediately, whether or not the
7   * image exists. When this applet attempts to draw the image on
8   * the screen, the data will be loaded. The graphics primitives
9   * that draw the image will incrementally paint on the screen.
10  *
11  * @param url An absolute URL giving the base location of the image.
12  * @param name The location of the image, relative to the url argument.
13  * @return The Image at the specified URL.
14  * @see Image
15  */
16  public Image getImage(URL url, String name) {
```

```

17     try {
18         return getImage(new URL(url, name));
19     } catch (MalformedURLException e) {
20         return null;
21     }
22 }

```

Generated HTML documentation:

```

getImage
public Image getImage(URL url,
                      String name)
Returns an Image object that can then be painted on the screen. The url argument must specify an absolute URL. The name argument is a
specifier that is relative to the url argument.

This method always returns immediately, whether or not the image exists. When this applet attempts to draw the image on the screen, the data will
be loaded. The graphics primitives that draw the image will incrementally paint on the screen.

Parameters:
url - an absolute URL giving the base location of the image.
name - the location of the image, relative to the url argument.

Returns:
the image at the specified URL.

See Also:
Image

```

Tooltip generated by IntelliJ IDE:



The screenshot shows a tooltip window titled "Documentation for getImage(URL, String)". It contains the following text:

```

JavaDocs
public Image getImage(URL url,
                      String name)

Returns an Image object that can then be painted on the screen. The url
argument must specify an absolute URL. The name argument is a specifier
that is relative to the url argument.

This method always returns immediately, whether or not the image exists.
When this applet attempts to draw the image on the screen, the data will
be loaded. The graphics primitives that draw the image will incrementally
paint on the screen.

Parameters:
    url - an absolute URL giving the base location of the image
    name - the location of the image, relative to the url argument

Returns:
    the image at the specified URL

See Also:
    Image

```

How ⚡⚡⚡

In the absence of more extensive guidelines (e.g., given in a coding standard adopted by your project), you can follow the two examples below in your code.

A minimal Javadoc comment example for methods:

```

1  /**
2   * Returns Lateral Location of the specified position.
3   * If the position is unset, NaN is returned.
4   *
5   * @param x X coordinate of position.
6   * @param y Y coordinate of position.
7   * @param zone Zone of position.
8   * @return Lateral Location.

```

```
9  * @throws IllegalArgumentException If zone is <= 0.  
10 */  
11 public double computeLocation(double x, double y, int zone)  
12     throws IllegalArgumentException {  
13     // ...  
14 }
```

A minimal JavaDoc comment example for classes:

```
1 package ...  
2  
3 import ...  
4  
5 /**  
6  * Represents a Location in a 2D space. A Point object corresponds to  
7  * a coordinate represented by two integers e.g., 3,6  
8  */  
9 public class Point {  
10     // ...  
11 }
```